AI Nutrition Tracker

Technical System Documentation

Mobile application • Local data layer • Serverless API • Gemini image analysis

Document purpose	Describe the application architecture, technology stack, screen responsibilities, user navigation flows, data handling, backend deployment, and AI integration.
Documentation baseline	Repository READMEs and configuration/source files reviewed for the mobile and API repositories, plus current project context.
Mobile repository	https://github.com/JiaSeng02/AI-Nutrition-Tracker
Backend repository	https://github.com/JiaSeng02/AI-Nutrition-Tracker-API
Production API	https://ai-nutrition-tracker-api-phi.vercel.app
Current status	Standalone Android APK built and reported working by the project owner.

Security note: this document explains where the Gemini API key belongs, but intentionally does not contain the key itself. Secrets must remain in Vercel environment variables and must never be committed to Git.

Contents

1. Executive summary
2. System architecture
3. Technology specifications
4. Application structure and code responsibilities
5. Screen inventory and page responsibilities
6. User flows and navigation
7. Data model and persistence
8. AI food analysis flow
9. Backend API specification
10. Vercel deployment and Gemini API key management
11. Build, release, and local development
12. Error handling, limitations, and security considerations
13. Maintenance checklist

Appendix A. Important URLs and commands

1. Executive summary

AI Nutrition Tracker is a personal-use, mobile-first food diary. It helps users record meals, review daily nutrition, manage a food catalog, maintain basic health measurements and targets, and estimate meal nutrition from a photo.

The mobile client is built with React Native, Expo SDK 57, TypeScript, and Expo Router. On native Android, SQLite stores diary entries and profile-related data locally on the device. The app does not require a user account or a cloud database for ordinary manual logging.

Photo-based analysis is the exception to the local-first design: the selected image is sent over the internet to a Vercel-hosted serverless API, which calls Google's Gemini API and returns a structured estimate. The user can review and edit that estimate before saving it locally.

The project has two separate GitHub repositories: the mobile application and the serverless API. They can be maintained and deployed independently.

Design principles

• **Local-first:** manual food logging and viewing saved entries do not depend on the backend.

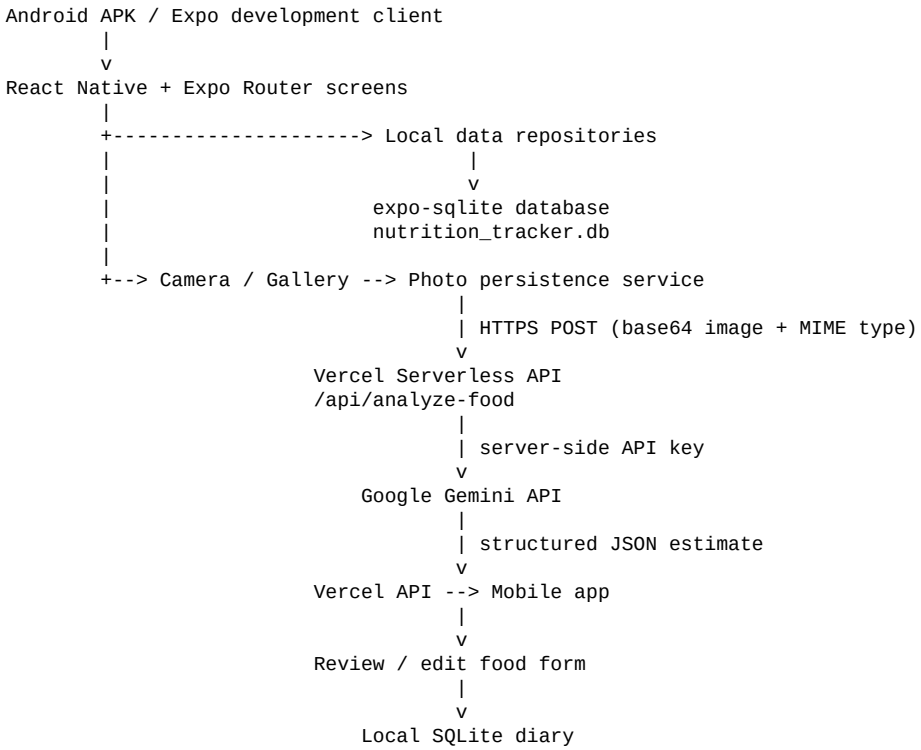
• **Human review:** AI output is an estimate and is presented for review/editing before it becomes a diary entry.

• **Separation of concerns:** screens, reusable components, repositories, services, types, and utilities have separate responsibilities.

• **Secret isolation:** Gemini credentials are stored server-side in Vercel, not inside the APK.

• **Personal-use scope:** current backend documentation says authentication and a server database are outside the current scope.

2. System architecture



The mobile app and API are separate deployables. Vercel is not the app's diary database: it hosts API functions. Gemini performs image interpretation; the app owns the local diary and review process.

System boundaries

Component	Responsibility	Needs internet?
Mobile UI	Screens, forms, charts, navigation, validation and user actions.	No for ordinary local interactions
Local data layer	Diary, food library, health measurements, targets, settings and export.	No for native local storage
Photo capture/storage	Capture/select a meal photo and persist it for app use.	Capture/select can be local
Vercel API	Validate requests, read server-side key, call Gemini and return JSON.	Yes
Gemini API	Interpret image and estimate serving and nutrition values.	Yes

3. Technology specifications

Layer	Technology / configuration	Role
Mobile framework	React Native 0.86.3; Expo SDK ~57.0.29	Cross-platform app runtime and native modules
Language	TypeScript ~6.0.3	Typed application code
Navigation	Expo Router ~57.0.27; main: expo-router/entry	File-based routing and tab navigation
Database	expo-sqlite ~57.0.4	Native local database; DB name nutrition_tracker.db
Image capture	expo-camera ~57.0.6	Capture meal photos
Image selection	expo-image-picker ~57.0.20	Choose an existing photo
Image display	expo-image ~57.0.5	Image rendering and preview UI
Photo persistence	expo-file-system ~57.0.7	Persist selected/captured images in app document storage
UI / motion	@expo/ui ~57.0.24; react-native-reanimated 4.5.1	Native UI support and animation
Safe layout	react-native-safe-area-context ~5.7.0	Safe-area handling
Backend runtime	Vercel serverless functions; TypeScript	HTTP API endpoints
AI provider	Google Gemini API; model gemini-3.5-flash-lite in source	Image-based food/nutrition estimation
Hosting	Vercel	Production backend hosting
Source control	GitHub, separate mobile and API repositories	Version control and project documentation
Android distribution	EAS Build preview profile configured for APK (per build workflow)	Build and distribute installable Android package

Version note: versions above are taken from package.json files reviewed for this document. Re-check repositories before future releases because dependency versions can change.

4. Application structure and code responsibilities

```
src/
  app/
    (tabs)/          Home, Diary, Health, Scan, Profile
    food/            Food add/edit form and entry detail routes
    food-library/    Food catalog routes
    profile/         Target-editing routes
    scan/            Photo review and AI analysis routes
  components/        Reusable forms, cards, charts and photo UI
  constants/         Theme and design tokens
  database/          Database implementations and repositories
  hooks/             Shared React hooks
  services/          Image persistence and AI-analysis client
  types/             Food, nutrition, health and target types
  utils/             Date, health and nutrition calculation helpers
```

Key data modules

Module	Responsibility
foodRepository.ts	Diary entries, entry changes and daily nutrition summaries.
foodLibraryRepository.ts	Built-in/custom food catalog and library management.
healthRepository.ts	Health profile, measurements and nutrition-trend data.
nutritionTargetRepository.ts	Target persistence, target sources and health-based suggestions.
settingsRepository.ts	Preferences, JSON export and clear-data actions.
database.native.ts	Native-platform SQLite implementation.
database.web.ts	Browser storage implementation for the web target.
capturedImageStore.ts (if present in current tree)	Shared image references/data used in capture-to-preview; confirm exact current implementation.
foodAnalysis.ts	Client-side request to food-analysis backend and response handling.

The screen layer should call repositories/services rather than embedding SQL or API details directly in UI components.

5. Screen inventory and page responsibilities

Main tab navigation

Screen / area	What it does	Typical user actions
Home	Dashboard for the current day. Shows daily calories/macronutrients against targets and summarizes meals by meal type.	Review today's progress; jump to add food or scan a meal.
Diary	Date-oriented food history, grouped by meal type, with nutrition totals and saved entries.	Change date; open entry details; edit or delete an entry.
Health	Basic health information and history, including BMI and recent measurements/trends where data is available.	Review profile measurements and trend visualizations.
Scan	Starts the meal-photo workflow using camera or gallery.	Take a photo or choose an image; proceed to preview.
Profile	Profile name, units, targets, settings and data-management actions.	Update profile; review/edit targets; export JSON; clear data after confirmation.

Food-related routes

Route / page type	Purpose
Food add/edit form	Create or edit a diary item. Supports food name, meal type, serving details, calories, protein, carbohydrates, fat, fiber, notes and optional photo. Numeric values are validated before save.
Food-entry details	Show a saved diary item, including serving details, nutrition, photo, notes and timestamp; provide edit/delete actions.
Food library	Search and filter catalog items; choose a catalog food to add to the diary; manage custom library records.
Profile target editor	Edit daily calorie and macronutrient targets, keeping default, suggested and custom target sources distinct.

Scan-related routes

Stage	Purpose
Capture/select	Use the camera or photo library to select a meal image.
Photo preview	Review the selected photo before analysis; allow retake or return if unsuitable.
Analysis loading/result	Send the photo to the API, receive food identification and nutrition estimates, and handle failures.
Review in food form	Pre-fill editable food fields with the estimate; correct food name, portion or nutrition before saving.

Exact route filenames and optional intermediate states can change as the app evolves. The current Expo Router file tree is the source of truth for exact paths.

6. User flows and navigation

6.1 App launch

```
Launch APK
  -> Expo Router root layout initializes app/database
  -> Main tab navigation
  -> Home loads today's summary from local repositories
```

6.2 Manual food entry

```
Home / Diary / Food Library
  -> Open Add Food or choose a library item
  -> Enter/review food name, meal type, serving and nutrition
  -> Validate fields
  -> Save via foodRepository
  -> Return to relevant screen
  -> Refresh summary / diary data
```

6.3 Edit or delete a diary entry

```
Diary -> Select date and entry -> Entry detail
  -> Edit -> form -> save update -> refresh list and totals
  -> Delete -> confirmation (where provided) -> remove -> refresh
```

6.4 Scan a meal with AI

```
Scan tab
  -> Camera capture OR gallery selection
  -> Photo preview
  -> Confirm image
  -> Mobile POSTs base64 image + MIME type to Vercel
  -> Vercel validates request and calls Gemini
  -> API returns structured estimate
  -> App opens/repopulates food form
  -> User reviews/edits estimate
  -> Save -> local diary
```

6.5 Profile, health and targets

```
Profile / Health
  -> Enter/update supported profile fields
  -> Save to local repository
  -> Health helpers calculate applicable indicators/estimates
  -> Review measurements, trends and targets
  -> Keep custom target or review/apply suggested target
```

6.6 Export and clear data

• Export: profile/settings/food data is serialized as JSON and shared using the device share sheet.

• Clear: the user confirms the clear-data action; local food records, health information and settings are removed according to the implementation.

• A JSON export is a user-managed backup and may contain personal health and food diary data.

7. Data model and persistence

Local-first storage

On native Android, SQLite is the durable local store. The database is named nutrition_tracker.db and is held in the app's private storage. The app does not currently document cloud synchronization or a server-side user database.

Data category	Examples / purpose	Expected persistence
Diary entries	Food name, meal type, serving, nutrition values, notes, timestamp and optional photo URI.	Local database
Food library	Built-in and custom catalog records used to speed up entry.	Local database / repository
Health profile	Age, height, weight, activity level and energy-estimation parameter.	Local database
Measurements/trends	Recent measurements and historical values used by health/trend screens.	Local database
Nutrition targets	Default, suggested and custom calorie/macronutrient targets.	Local database
Settings	Display name, units and other preferences.	Local database
Photos	Files saved under app document directory in food_photos/; diary records keep photo URI.	App-private file storage

Important storage behavior

• Manual logging is intended to work offline because it uses local storage.

• AI photo analysis requires network access and a reachable backend/Gemini API.

• Expo Go and the standalone APK have separate application storage sandboxes; Expo Go data will not automatically migrate to the APK.

• Uninstalling the app may remove its private local database and photo files. Export JSON before risky changes or uninstalling.

• Web uses a separate browser-storage implementation; web data is not the same database as Android SQLite.

Health calculations

The codebase documents basic BMI and estimated daily energy needs. The energy estimate uses the Mifflin–St Jeor equation plus an activity factor, and the current utility returns an estimate only for age 18 or above. Health outputs are informational, not medical advice. AI nutrition values are estimates, not measured facts.

8. AI food analysis flow

Request lifecycle

• The user captures a photo or selects one from the gallery.

• The app prepares the image for the API client and supplies its MIME type.

• The app POSTs JSON to https://ai-nutrition-tracker-api-phi.vercel.app/api/analyze-food.

• The backend checks HTTP method, GEMINI_API_KEY, required image fields, supported MIME type and image length.

• The backend calls Gemini generateContent with image bytes and a prompt requesting structured food and nutrition estimates.

• Gemini returns JSON; the backend parses it and wraps it in an API response.

• The app displays the result for review; the user can edit it before local save.

Output fields

Field	Meaning
foodName	Concise identified dish or food name.
confidence	Food-identification confidence score from 0 to 1.
servingEstimate / servingUnit	Estimated quantity and unit such as plate, bowl, piece, slice, cup or serving.
calories	Estimated energy in kcal for the identified serving.
protein / carbs / fat / fiber	Estimated macronutrients and fiber, in grams.
notes	Short uncertainty note or important assumptions.

The prompt instructs Gemini not to invent precise ingredients that cannot reasonably be inferred from the photo and to use low confidence if the image does not clearly contain food. Estimates can still be inaccurate because photos do not reliably reveal all ingredients, cooking methods or exact portion mass.

9. Backend API specification

Backend repository: https://github.com/JiaSeng02/AI-Nutrition-Tracker-API

Production base URL: https://ai-nutrition-tracker-api-phi.vercel.app

Endpoint	Method	Purpose
/api/health	GET	Simple health check; returns status ok and service name.
/api/analyze-food	POST	Accepts a base64 image and MIME type; calls Gemini and returns structured food/nutrition estimates.

POST request

```
POST /api/analyze-food
Content-Type: application/json

{
  "image": "<base64 image data, without a data URL prefix>",
  "mimeType": "image/jpeg"
}
```

Supported image MIME types: image/jpeg, image/png, image/webp. The source sets a maximum base64 string length of 3,500,000 characters. The endpoint expects POST; other methods return HTTP 405.

Successful response shape

```
{
  "success": true,
  "estimated": true,
  "model": "gemini-3.5-flash-lite",
  "analysis": {
    "foodName": "...",
    "confidence": 0.0,
    "servingEstimate": 1,
    "servingUnit": "serving",
    "calories": 0,
    "protein": 0,
    "carbs": 0,
    "fat": 0,
    "fiber": 0,
    "notes": "..."
  }
}
```

Errors and status codes

Status / error	Meaning / typical cause
400 Invalid Request	image or mimeType is missing or empty.
400 Invalid Image Type	MIME type is not JPEG, PNG or WebP.
405 Method Not Allowed	The endpoint was called with an unsupported HTTP method.
413 Image Too Large	Base64 image string exceeds configured length limit.
500 Server Configuration Error	GEMINI_API_KEY is not configured in deployment environment.
Gemini API Error	Google API returned a non-success status; backend forwards status/message where available.
502 Invalid Gemini Response	No text result or invalid JSON was returned by Gemini.
500 Analysis Failed	Unexpected error during the request.

The backend README records that the production health endpoint returned HTTP 200 and an empty analysis request returned HTTP 400 during a documented check on 15 October 2026. A valid image request also requires a valid configured Gemini key.

10. Vercel deployment and Gemini API key management

What Vercel does

Vercel hosts the TypeScript API endpoints as serverless functions. It accepts requests from the mobile app, validates them, makes the outbound Gemini request, and returns JSON. Vercel does not store the diary or profile in the current architecture.

What the Gemini API key does

GEMINI_API_KEY authorizes the backend to call Google's Gemini API. The mobile app should never contain this secret. Backend code reads it from process.env.GEMINI_API_KEY.

Environment variable setup

- Open the Vercel dashboard and select the AI Nutrition Tracker API project.
- Open Project Settings → Environment Variables.
- Create/update the variable named GEMINI_API_KEY with the API key value.
- Select the deployment environments where the key should be available.
- Save the variable and redeploy the relevant environment so functions use the updated configuration.
- Do not place the real key in README files, mobile source, screenshots, issue reports or Git commits.

The key value is deliberately omitted. If a key has ever been committed or exposed in logs, revoke/rotate it through the provider and update the Vercel environment variable.

Deploying backend changes

```
# From the backend repository directory
npm install
npm run typecheck

# If the Vercel CLI is installed and the project is linked:
vercel deploy
# Production deployment (when intentionally releasing):
vercel --prod
```

Use the workflow configured for the Vercel project; these are examples. A successful deployment does not by itself prove Gemini analysis works. Test health and a valid image request.

Security limitations to address before broader use

The current backend README says authentication is outside the present scope. The reviewed handler validates requests and limits image size, but no application-level authentication or rate limiting is visible. Because the endpoint is publicly reachable, an unrelated caller could attempt to consume Gemini quota. Consider access control, rate limiting, monitoring and quota alerts before sharing the APK widely.

11. Build, release, and local development

Mobile development

```
cd "D:\USER\GitHub\AI Nutrition Tracker"
npm install
npx expo start
```

Use the QR code with Expo Go for development testing. Run checks before building:

```
npx expo-doctor
npx expo install --check
npm run lint
npx tsc --noEmit
```

Android APK via EAS Build

The project owner reports that the standalone APK has been built and works. The documented preview-profile approach is:

```
npm install -g eas-cli
eas login
eas whoami

# eas.json preview profile should specify internal distribution
# and android.buildType: apk
eas build --platform android --profile preview
```

EAS cloud build produces an APK download link. Install it directly on the Android device or use ADB if installed and USB debugging is enabled. Keep Android signing credentials safe so future builds can update the same app.

Important release checks

• Keep expo.android.package stable: current app.json value is com.ainutritiontracker.app.

• Confirm the production API URL is used in the installed build; localhost points to the device itself.

• Test camera/gallery, preview, AI request, result review, local save, app restart and network error handling on the standalone APK.

• Remember the APK has its own local database, separate from Expo Go.

• If changing native dependencies, rebuild the APK; JavaScript-only changes depend on the chosen release/update mechanism.

12. Error handling, limitations, and security considerations

Operational limitations

• AI analysis requires internet access, Vercel availability and a working Gemini key/quota.

• Manual logging is designed to work without the backend.

• Nutrition results are estimates; user review is part of the intended workflow.

• No account system, cloud sync or server-side diary database is documented in the current implementation.

• Local data can be lost if the app is uninstalled or local app data is cleared; export JSON before risky changes.

• Expo Go storage does not migrate automatically to the standalone APK.

Security and privacy

• Keep GEMINI_API_KEY server-side in Vercel environment variables.

• Images selected for AI analysis leave the device and are sent to the configured API and Gemini. Avoid scanning images that must remain entirely on-device.

• The local-first design reduces backend transfer of diary/profile data, but scanning a photo intentionally sends that image over the network.

• Add authentication/rate limiting if the endpoint or APK is shared beyond trusted personal use.

• Avoid logging raw images, API keys or personal diary/health data.

• Use JSON export carefully; it may include sensitive personal information.

Health information disclaimer

BMI and estimated energy needs are informational calculations and not a diagnosis. Photo-generated nutrition values can be wrong. The app is not a substitute for qualified medical or nutrition advice.

13. Maintenance checklist

■ Before mobile changes: inspect git status and review diffs.

■ Install dependencies with npm install and commit intentional package.json/package-lock.json changes together.

■ Run npx expo-doctor, npx expo install --check, npm run lint and npx tsc --noEmit.

■ For backend changes: run npm install and npm run typecheck.

■ After backend deployment: test GET /api/health and a valid POST /api/analyze-food request.

■ Check Vercel environment variables if analysis fails with a server configuration error.

■ Monitor Gemini quota and consider endpoint rate limiting.

■ Build and test a new APK after native dependency or configuration changes.

■ Export local data before uninstalling or clearing app data.

■ Update this document when routes, dependencies, API contracts, storage design or deployment URLs change.

Appendix A. Important URLs and commands

Resource	Value
Mobile GitHub repository	https://github.com/JiaSeng02/AI-Nutrition-Tracker
Backend GitHub repository	https://github.com/JiaSeng02/AI-Nutrition-Tracker-API
Production API base	https://ai-nutrition-tracker-api-phi.vercel.app
Health check	https://ai-nutrition-tracker-api-phi.vercel.app/api/health
Food analysis endpoint	https://ai-nutrition-tracker-api-phi.vercel.app/api/analyze-food
Mobile start	npx expo start
Mobile checks	npx expo-doctor; npx expo install --check; npm run lint; npx tsc --noEmit
Backend typecheck	npm run typecheck
APK build	eas build --platform android --profile preview

Documentation note: this guide is based on repository files reviewed for this version and project details available at generation time. Confirm current source files—especially the Expo Router tree, database schema/migrations and EAS configuration—before making implementation-sensitive changes.